

Synthese Java Theorie

Chapitre 1

Java est un langage de programmation orienté objet, simple et moderne, créé par Sun Microsystems en 1995, aujourd'hui propriété d'Oracle.

Il permet d'écrire une fois un programme et de l'exécuter partout grâce à la machine virtuelle Java (JVM), qui assure la portabilité sur toutes les plateformes (« write once, run everywhere »).

Le code source (.java) est compilé en bytecode (.class), interprété ensuite par la JVM, ce qui rend l'exécution indépendante du système d'exploitation.

Java supprime les pointeurs, l'héritage multiple et la gestion manuelle de la mémoire pour plus de sécurité et de simplicité.

Les programmes Java sont structurés autour des concepts de classes, objets, encapsulation, héritage et polymorphisme.

On utilise des packages pour organiser le code, et les exceptions pour gérer les erreurs¹.

Java permet la création d'applications console, graphiques et la connexion à des bases de données.

Il est très utilisé pour sa robustesse, sa simplicité, sa portabilité et sa sécurité.

Structure

```
java

public class bonjour {
    public static void main(String args[]) {
        System.out.println("Bonjour Monde !");
    }
}
```

Exemple de question

Quels sont les avantages de Java par rapport à d'autres langages de programmation ?

Comment la machine virtuelle Java facilite-t-elle le développement multiplateforme ?

Quelles sont les principales phases de la vie d'un code Java ?

Comment Java gère-t-il les exceptions et les erreurs ?

Quels sont les principaux concepts de la programmation orientée objet en Java ?

Chapitre 2 :

1. Classes et Objets

- **Classe** : Modèle ou plan qui définit les attributs (variables) et les comportements (méthodes) des objets.

- Exemple :

- java

```
public class Personne {  
    private String nom; // Attribut  
    public void setNom(String nom) { this.nom = nom; } // Méthode  
}
```

- **Objet** : Instance d'une classe, créée avec `new`.

- Exemple :

- java

```
Personne p = new Personne();
```

- **Constructeurs** : Méthodes spéciales qui initialisent les objets, avec possibilité de surcharge (polymorphisme).
- **Méthodes** : Fonctions définies dans une classe, pouvant être publiques, privées, protégées ou par défaut (package).
- **Mots-clés** :
 - `this` : Référence à l'objet courant.
 - `static` : Variable ou méthode de classe (partagée par toutes les instances).

2. Héritage

- **Principe** : Une classe (enfant) hérite des attributs et méthodes d'une autre classe (parent) via `extends`.

- Exemple :

- java

```
public class Employee extends Personne { ... }
```

- **Mot-clé** `super` :
 - Permet d'appeler le constructeur ou une méthode de la classe parente.
 - Exemple :
 - java

```
super(nom, age); // Appel du constructeur parent
```

- **Redéfinition** (`@Override`) : Permet de modifier une méthode héritée (exemple : `toString()`).
- **Classes abstraites** (`abstract`) :
 - Ne peuvent pas être instanciées directement.
 - Peuvent contenir des méthodes abstraites (sans implémentation).

3. Packages

- **But** : Organiser les classes et éviter les conflits de noms.
- **Syntaxe** :java

```
package hepl.java.basics;
```

- **Règles** :
 - Un package correspond à un répertoire.
 - Une classe `public` doit être dans un fichier du même nom (`Personne.java` pour `public class Personne`).
- **Import** :java

```
import hepl.java.basics.Personne; // Importer une classe
import hepl.java.basics.*;        // Importer tout le package
```

4. Classpath

- **Rôle** : Indique l'emplacement des fichiers `.class` (chemins des répertoires/jar).
- **Définition** :
 - Variable d'environnement `CLASSPATH` .
 - Option de ligne de commande :

```
java -classpath "chemin1;chemin2" MaClasse
```

- **Exemple** :
 - Pour `hepl.java.basics.Personne` , le fichier `Personne.class` doit être dans `hepl/java/basics/` .

5. Interfaces (Important !)

- **Définition** : Contrat spécifiant des méthodes abstraites (sans implémentation) et des constantes.

- Exemple :

```
public interface Identifiable {  
    String getId(); // Méthode abstraite  
}
```

- **Implémentation** : Une classe utilise `implements` pour respecter le contrat.

- Exemple :
- java

```
public class Personne implements Identifiable { ... }
```

- **But** :

- **Abstraction** : Définir un comportement commun pour des classes non liées.
- **Polymorphisme** : Manipuler des objets hétérogènes via une interface.
- java

```
Identifiable[] objets = { new Personne(), new Facture() };
```

- **Comparaison avec classe abstraite** :

- **Interface** :

- Pas d'attributs (sauf constantes).
- Pas de constructeur.
- Héritage multiple possible (`implements A, B`).

- **Classe abstraite** :

- Peut avoir des attributs et méthodes implémentées.
- Constructeur possible.
- Héritage simple uniquement (`extends`).

6. Exceptions

- **Gestion** :

- `try` : Bloc de code à surveiller.
- `catch` : Traite l'exception.
- `finally` : S'exécute dans tous les cas.
- `throw` : Lance une exception.

- **Hiérarchie :**

- `Throwable` → `Error` (erreur grave) / `Exception` (à traiter).
- `RuntimeException` : Exception non vérifiée (ex : division par zéro).

- **Exemple :java**

```
try {  
    int x = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Erreur : " + e.getMessage());  
}
```

7. Piège de l'Égalité et Clonage

- **Égalité :**

- `==` : Compare les références (adresses mémoire).
- `equals()` : Compare le contenu des objets (à redéfinir).
- java

```
Personne p1 = new Personne("Alice");  
Personne p2 = new Personne("Alice");  
System.out.println(p1 == p2); // false  
System.out.println(p1.equals(p2)); // true si redéfini
```

- **Clonage :**

- Nécessite d'implémenter `Cloneable` et de redéfinir `clone()` .
- java

```
public class Personne implements Cloneable {  
    @Override  
    public Object clone() {  
        return new Personne(this.nom, this.age);  
    }  
}
```

8. Classes Types Primitifs (Wrapper Classes)

- **But** : Encapsuler les types primitifs (`int`, `double`, etc.) en objets.
 - Exemple : `Integer`, `Double`, `Boolean` .
- **Utilisation** :
 - Conversion :

- java

```
int i = Integer.parseInt("123");
```

- Collections : Les génériques nécessitent des objets (`List<Integer>`).
- **Auto-boxing** : Conversion automatique entre type primitif et [wrapper.java](#)

```
Integer j = 10; // Auto-boxing  
int k = j;      // Unboxing
```

Résumé des Points Clés

- **Classes/Objets** : Structure fondamentale en Java avec encapsulation.
- **Héritage** : Réutilisation et modification du comportement d'une classe parente.
- **Packages/Classpath** : Organisation et localisation des classes.
- **Interfaces** : Contrats permettant un polymorphisme flexible.
- **Exceptions** : Gestion structurée des erreurs.
- **Égalité/Clonage** : Distinction entre référence et contenu ; clonage via `Cloneable` .
- **Wrappers** : Utilisation des types primitifs comme objets.

Chapitre 3

1. Principe Général de la Programmation Fenêtrée

- **Swing** est une bibliothèque Java pour créer des interfaces graphiques (GUI).
- **Hierarchie des composants** :
 - `Component` → `Container` → `Window` → `Frame` / `Dialog` → `JFrame` / `JDialog` .
 - Les composants Swing (ex: `JButton` , `JLabel`) héritent de `JComponent` .
- **Indépendance plateforme** : Les composants Swing sont écrits en Java pur, contrairement à AWT.

2. Layouts (Gestionnaires de Disposition)

- **Rôle** : Organiser les composants dans un conteneur (`JPanel` , `JFrame` , etc.).
- **Types principaux** :
 - `FlowLayout` : Aligne les composants en ligne (par défaut pour `JPanel`).
 - `GridLayout` : Dispose les composants dans une grille (lignes × colonnes).

- **BorderLayout** : Répartit les composants aux points cardinaux (par défaut pour **JFrame**).
- **BoxLayout** : Aligne les composants verticalement ou horizontalement.
- **Exemple** :CopyDownload
- java

```
JPanel panel = new JPanel(new GridLayout(2, 2));
panel.add(new JButton("1"));
panel.add(new JButton("2"));
```

3. Différence entre **JFrame** , **JDialog** et **JPanel**

- **JFrame** : Fenêtre principale d'une application.CopyDownload
- java

```
JFrame frame = new JFrame("Titre");
frame.setSize(400, 300);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

- **JDialog** : Fenêtre modale ou non modale pour les dialogues.CopyDownload
 - **Modale** : Bloque l'interaction avec la fenêtre parente jusqu'à sa fermeture.
- java

```
JDialog dialog = new JDialog(parentFrame, "Dialogue", true); // Modal
```

- **JPanel** : Conteneur intermédiaire pour regrouper des composants.
 - Souvent utilisé comme **contentPane** d'une **JFrame** .

4. Gestion des Événements Graphiques

- **Mécanisme** :
 - a. **Source** : Composant générant l'événement (ex: **JButton**).
 - b. **Listener** : Objet qui implémente une interface pour traiter l'événement.
 - c. **Enregistrement** : Lier le listener à la source via **addXXXListener()** .
- **Interfaces courantes** :
 - **ActionListener** : Clic sur un bouton (**actionPerformed**).
 - **MouseListener** : Interactions souris (**mouseClicked** , **mouseEntered** , etc.).
 - **WindowListener** : Gestion des fenêtres (**windowClosing** , **windowOpened**).

- **Exemple** :CopyDownload

- java

```
button.addActionListener(e -> System.out.println("Bouton cliqué !"));
```

5. Composants Graphiques Courants

- **JButton** : Bouton cliquable.
- **JLabel** : Texte ou icône.
- **TextField** / **PasswordField** : Saisie de texte.
- **CheckBox** / **RadioButton** : Cases à cocher (exclusives avec **ButtonGroup**).
- **ComboBox** : Liste déroulante.
- **JList** / **JTable** / **JTree** : Affichage de données complexes.

6. Menus

- **JMenuBar** : Barre de menus.
- **JMenu** : Menu déroulant (ex: "Fichier").
- **JMenuItem** : Élément de menu (ex: "Ouvrir").

- **Exemple** :CopyDownload

- java

```
JMenuBar menuBar = new JMenuBar();
```

- JMenu m

```
nu = new JMenu("Fichier");  
menu.add(new JMenuItem("Ouvrir"));  
menuBar.add(menu);  
frame.setJMenuBar(menuBar);
```

7. Boîtes de Dialogue

- **JOptionPane** : Boîtes standard (message, confirmation, saisie).CopyDownload

- java

```
JOptionPane.showMessageDialog(frame, "Message");  
int choix = JOptionPane.showConfirmDialog(frame, "Confirmer ?");
```

- **JFileChooser** : Sélection de fichiers.CopyDownload
- java

```
JFileChooser chooser = new JFileChooser();
if (chooser.showOpenDialog(frame) == JFileChooser.APPROVE_OPTION) {
    File fichier = chooser.getSelectedFile();
}
```

8. Modèle MVC (Modèle-Vue-Contrôleur)

- **Modèle** : Gère les données (ex: **DefaultTableModel** pour **JTable**).
- **Vue** : Affiche les données (ex: **JTable**).
- **Contrôleur** : Gère les interactions (ex: **ActionListener**).
- **Exemple avec JTable** :CopyDownload
- java

```
String[] colonnes = {"Nom", "Âge"};
Object[][] données = {{ "Alice", 25}, {"Bob", 30}};
DefaultTableModel model = new DefaultTableModel(données, colonnes);
JTable table = new JTable(model);
```

9. Points à ne pas Approfondir

- **Renderers/Editors pour JTable** : Personnalisation avancée de l'affichage/édition.
- **MVVM** : Variante de MVC (non couverte ici).

Conclusion

- **Priorités** :
 - Comprendre les layouts, la gestion des événements, et les composants de base.
 - Maîtriser **JFrame**, **JDialog**, **JPanel**, et les boîtes de dialogue standards.
 - S'appuyer sur le modèle MVC pour les interfaces complexes (ex: **JTable**).
- **À éviter** : Les détails des méthodes spécifiques à chaque composant.

Chapitres 4

1. Signification des principes SOLID

- **S - Single Responsibility Principle (SRP)**

- □ Chaque classe doit avoir une seule responsabilité.
- □ Ex : Séparer `Employee` (calcul salaire) et `DataAccessEmployee` (sauvegarde).

- **O - Open/Closed Principle (OCP)**

- □ Une classe doit être **ouverte à l'extension**, mais **fermée à la modification**.
- □ Ex : Utiliser une interface `Shape` pour `Rectangle` et `Square`, sans modifier les classes existantes.

- **L - Liskov Substitution Principle (LSP)**

- □ Une sous-classe doit pouvoir remplacer sa classe parente sans briser le comportement du programme.
- □ Ex : Faire une interface `FlyingCapacity` pour les oiseaux qui volent, au lieu de forcer tous les oiseaux à voler.

- **I - Interface Segregation Principle (ISP)**

- □ Il vaut mieux plusieurs petites interfaces qu'une grosse inutilisable pour certains.
- □ Ex : `Workable` pour travailler, `Eatable` pour manger. Les robots implémentent seulement `Workable`.

- **D - Dependency Inversion Principle (DIP)**

- □ Les classes haut niveau ne doivent pas dépendre des classes bas niveau, mais d'abstractions (interfaces).
 - □ Ex : `Switch` utilise une interface `Switchable`, pas directement `LightBulb`.
-

2. Modèle MVC (Model-View-Controller)

Composant	Rôle	Utilité
Model (M)	Gère les données et la logique métier.	Contient les entités (ex : <code>Article</code>) et accès aux données (<code>DAO</code>).
View (V)	Interface utilisateur.	Affiche les données et capte les interactions (ex : console, Swing GUI).
Controller (C)	Gère les événements et la coordination entre Model et View.	Réagit aux actions de l'utilisateur et modifie le modèle.

□ Le **but** de MVC : **séparer les responsabilités** pour une application plus modulable, testable et maintenable.

3. Comment MVC respecte SOLID

- **S (Single Responsibility)** : Chaque couche (M, V, C) a une seule fonction.
- **O (Open/Closed)** : Ajouter une nouvelle vue ou un nouveau modèle ne nécessite pas de modifier le contrôleur.
- **L (Liskov Substitution)** : Une nouvelle `View` ou `DAO` peut remplacer l'ancienne si elle implémente la bonne interface.

- **I (Interface Segregation)** : Les interfaces `DataAccessLayer` et `ViewArticles` sont ciblées, pas trop grosses.
 - **D (Dependency Inversion)** : Le contrôleur dépend d'abstractions (`ViewArticles` , `DataAccessLayer`) et reçoit ses dépendances par injection.
-

4. Rôle des Interfaces dans SOLID et MVC

- Permettent l'**extensibilité** (O), sans modification.
 - Assurent le **découplage** entre les classes (DIP).
 - Forcent à avoir des **interfaces cohérentes et petites** (ISP).
 - Dans MVC :
 - `DataAccessLayer` abstrait l'accès aux données (DAO en mémoire, en base de données...).
 - `ViewArticles` abstrait la vue (console, Swing...).
-

5. DAO et Programmation événementielle

- **DAO (Data Access Object)** :
 - □ C'est la couche qui gère l'accès aux données (ex : `DAOArticles` stocke des `Article` dans une `ArrayList`).
 - □ Permet de **séparer** la persistance (sauvegarde, récupération) de la logique métier.
- **Programmation événementielle** :
 - □ L'application réagit à des événements (clics, saisies...).
 - □ Le **Controller** implémente `ActionListener` et gère des événements comme "Ajouter", "Modifier", "Supprimer" un article. □ C'est l'**utilisateur** qui pilote le flux du programme par ses actions.

Chapitre 5

1. Notion générale de Conteneur et Collections en Java

- **Conteneur** = structure qui stocke des éléments du même type de façon organisée.
- Exemples : `Vector` , `ArrayList` , `LinkedList` , `Stack` , `Hashtable` ...
- En Java, les conteneurs sont regroupés dans le **package** `java.util` sous le nom de **collections**.

- Les collections implémentent souvent `Collection<T>` (liste, pile, file...) ou `Map<K, V>` (dictionnaire).
- ♦ Les conteneurs peuvent être parcourus via un **itérateur** (`Iterator<T>`) ou via la boucle `for-each`.

2. Algorithmes associés : Tri, Max, etc.

- Classe `Collections` : méthodes utilitaires comme `sort()`, `max()`, `min()`, `shuffle()`, etc.

Fonction	Description	Exemple
<code>Collections.sort(list)</code>	Trie selon l' ordre naturel	nécessite <code>Comparable<T></code>
<code>Collections.sort(list, comp)</code>	Trie selon un comparateur	nécessite <code>Comparator<T></code>
<code>Collections.max(list)</code>	Retourne le plus grand élément	
<code>Collections.min(list)</code>	Retourne le plus petit élément	

- Interface `Comparable<T>`
 - Définir un **ordre naturel** d'objets : méthode `compareTo(T o)`.
 - Exemple : trier des `Individu` par nom/prénom.
- Interface `Comparator<T>`
 - Définir un **ordre personnalisé** : méthode `compare(T o1, T o2)`.
 - Exemple : trier des `Individu` par âge.
- ♦ **Résumé :**
 - Utiliser `Comparable` = tri **interne** à la classe.
 - Utiliser `Comparator` = tri **externe**.

3. Dictionnaires en Java

- Dictionnaire = associer une **clé** à une **valeur** (`Map<K, V>`).

- Exemple de classe : `Hashtable<K, V>` (ancien, synchronisé).
 - Méthodes importantes :
 - `put(key, value)` □ insérer une association.
 - `get(key)` □ retrouver une valeur à partir d'une clé.
 - `remove(key)` □ supprimer une association.
 - `containsKey(key)` □ vérifier la présence d'une clé.
 - ♦ Exemple : associer des espions (`Espion`) à leur code secret.
-

4. Découpage des chaînes de caractères

- Classe `StringTokenizer`
 - Découpe une chaîne avec des **délimiteurs** (`:`, `/`, etc.). □ Méthodes : `hasMoreTokens()`, `nextToken()`.
 - Méthode `split(String regex)` de `String`
 - Découpe une chaîne avec une **expression régulière**. □ Retourne directement un **tableau de String** (`String[]`).
 - ♦ Exemple :
 - `StringTokenizer` □ plus manuel.
 - `split(" ")` □ plus simple pour découper selon un espace.
-

5. Indépendance vis-à-vis du système

- Classe `System` :
 - Permet de récupérer des informations sur l'utilisateur, l'OS, le répertoire courant, etc.
 - Exemple :
 - `System.getProperty("user.name")` □ Nom utilisateur
 - `System.getProperty("os.name")` □ Système d'exploitation
 - `System.getProperty("user.dir")` □ Répertoire de travail
 - ♦ C'est un principe clé en Java : **même code** fonctionne sur **Windows, Linux, Mac, etc.**
-

6. Classe `Properties`

- Structure clé/valeur spécialisée pour **charger, sauvegarder** et **manipuler** des

configurations (fichiers `.properties`).

- Méthodes principales :
 - `getProperty(key)` □ lire une valeur.
 - `setProperty(key, value)` □ définir une valeur.
 - `list(PrintStream out)` □ afficher toutes les propriétés.
 - `propertyNames()` □ obtenir toutes les clés.
- Utilisé avec :
 - □ **`System.getProperties()`** pour accéder aux propriétés système.
 - □ **Chargement depuis un fichier** (ex: `config.properties`).



Résumé rapide

Sujet	Outil principal	Exemple
Conteneurs	<code>Collection<T></code> , <code>Map<K, V></code>	<code>ArrayList</code> , <code>Hashtable</code>
Tri et Max/Min	<code>Collections</code> , <code>Comparable</code> , <code>Comparator</code>	Trier des <code>Individu</code>
Dictionnaire	<code>Hashtable</code> , <code>Map</code>	Associer <code>code</code> □ <code>Espion</code>
Découpage	<code>StringTokenizer</code> , <code>split()</code>	Découper une URL
Indépendance OS	<code>System.getProperty()</code>	Obtenir nom d'utilisateur
Fichiers de config	<code>Properties</code>	Charger des paramètres

1. Gestion des fichiers **Properties**

- Classe **Properties** = gestion d'un ensemble de couples **clé/valeur**.
- Utilisée pour :
 - Paramètres de configuration
 - Stockage de mots de passe, etc.
- Charger un fichier **.properties** :

```
java
CopierModifier
Properties prop = new Properties();
prop.load(new FileInputStream("config.properties"));
```

- Lire une propriété :

```
java
CopierModifier
String password = prop.getProperty("admin");
```

- Afficher toutes les propriétés :

```
java
CopierModifier
prop.list(System.out);
```

- Format du fichier :

```
ini
CopierModifier
```

```
user=admin  
password=1234
```

♦ **Extension usuelle** : `.properties`

2. Gestion des fichiers orientés "Data" : `DataInputStream` et `DataOutputStream`

- **But** : lire/écrire des données primitives Java (`int`, `float`, `String`, `boolean`, etc.) dans un fichier binaire.
- **Écriture** :

```
java  
CopierModifier  
DataOutputStream dos = new DataOutputStream(new FileOutputStream("datafile.dat"));  
dos.writeInt(123);  
dos.writeUTF("hello");  
dos.close();
```

- **Lecture** :

```
java  
CopierModifier  
DataInputStream dis = new DataInputStream(new FileInputStream("datafile.dat"));  
int n = dis.readInt();  
String s = dis.readUTF();  
dis.close();
```

♦ Utilisés pour **échanges fiables** entre programmes Java sans se soucier du format binaire bas niveau.

3. Gestion des fichiers sérialisés ("Objets persistants")

- **Sérialiser un objet** = sauvegarder son état complet sur disque pour le recharger plus tard.
- Classe `ObjectOutputStream` :

```
java  
CopierModifier  
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("objects.ser"));  
oos.writeObject(monObjet);  
oos.close();
```

- Classe `ObjectInputStream` :

```
java  
CopierModifier  
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("objects.ser"));
```

```
MonObjet obj = (MonObjet) ois.readObject();
ois.close();
```

- □ L'objet à sauvegarder doit implémenter l'**interface** `Serializable` (aucune méthode à implémenter).
- ♦ *Utilisation typique : sauvegarde d'états, cache, persistance entre sessions.*

4. Gestion des fichiers textes

- Pour **lire un fichier texte** ligne par ligne :

```
java
CopierModifier
BufferedReader br = new BufferedReader(new FileReader("texte.txt"));
String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
br.close();
```

- Pour **écrire dans un fichier texte** :

```
java
CopierModifier
BufferedWriter bw = new BufferedWriter(new FileWriter("texte.txt"));
bw.write("Bonjour");
bw.newLine();
bw.write("Tout le monde");
bw.close();
```

- Les classes importantes :
 - `FileReader` / `FileWriter` (accès simple)
 - `BufferedReader` / `BufferedWriter` (accès optimisé)
- ♦ ***newLine()** insère un retour à la ligne correct selon l'OS (Windows, Linux...).*

5. Notions de couches bas niveau et haut niveau

Couche	Description	Exemple
Bas niveau	Accès direct aux bytes ou caractères , traitement brut	<code>FileInputStream</code> , <code>FileOutputStream</code> , <code>InputStreamReader</code> , <code>OutputStreamWriter</code>

Haut niveau	Ajoute des fonctionnalités (bufferisation, traitement par lignes, gestion d'objets)	<code>BufferedReader</code> , <code>BufferedWriter</code> , <code>DataInputStream</code> , <code>ObjectInputStream</code>
--------------------	---	--

▣ Rôle de chaque couche :

- **Bas niveau** = accès physique brut aux fichiers (rapide mais basique).
- **Haut niveau** = facilite la vie du programmeur en **masquant les détails** et **optimisant les performances** (ex : lecture par blocs).



Résumé ultra rapide

Sujet	Classe principale	Type de fichier
Fichiers de propriétés	<code>Properties</code>	Texte (<code>.properties</code>)
Données primitives	<code>DataInputStream</code> , <code>DataOutputStream</code>	Binaire
Objets persistants	<code>ObjectInputStream</code> , <code>ObjectOutputStream</code>	Binaire
Fichiers textes	<code>BufferedReader</code> , <code>BufferedWriter</code>	Texte
Couches bas/haut niveau	<code>FileInputStream</code> vs <code>BufferedReader</code> , etc.	Tous

Chapitre 7

1. Création manuelle des JAR (librairie et exécutable)

- **But** : Rassembler des fichiers (`.class` , `.xml` , images, etc.) dans un fichier `.jar` pour faciliter le déploiement et la distribution d'une application ou d'une librairie Java.
- **Format** : Compression au format ZIP, avec possibilité d'ajouter une signature électronique.
- **Commandes** :

- **Création d'un JAR :**

```
jar cvf <nom-du-fichier.jar> <fichiers-ou-répertoires>
```

- **Exemple :**

```
jar cvf Applic.jar mTransports/*.class TestTransports.class
```

- **Mise à jour d'un JAR :**

```
jar uvf <nom-du-fichier.jar> <fichiers-à-ajouter>
```

- **Extraction :**

```
jar xvf <nom-du-fichier.jar>
```

- **Affichage du contenu :**

```
jar tf <nom-du-fichier.jar>
```

2. Fichier Manifeste (**MANIFEST.MF**)

- **Rôle :** Contient des métadonnées sur les fichiers encapsulés dans le JAR, notamment :

- **Main-Class :** Classe contenant la méthode `main()` pour les JAR exécutables.
- **Class-Path :** Liste des JAR nécessaires au fonctionnement de l'application.

- **Exemple de contenu :**

```
Manifest-Version: 1.0
Created-By: 1.8.0_60 (Oracle Corporation)
Main-Class: TestTransports
Class-Path: Transports.jar
```

- **Création manuelle :**

- Créer un fichier `manifeste.txt` avec les métadonnées.
- Utiliser l'option `m` pour inclure le manifeste dans le JAR :

```
jar cvfm Applic.jar manifeste.txt mTransports/*.class TestTransports.class
```

3. Exécution d'un JAR

- **JAR exécutable :**

```
java -jar Applic.jar
```

- **Prérequis :** Le fichier manifeste doit contenir l'attribut `Main-Class`.

- **Problèmes courants :**

- `NoClassDefFoundError` : Indique que des dépendances sont manquantes. Résolu en

ajoutant `Class-Path` dans le manifeste.

- `UnsupportedClassVersionError` : La version de la JVM est inférieure à celle utilisée pour la compilation.

4. ClassPath d'un JAR

- **Définition** : Liste des chemins où la JVM recherche les classes et les ressources.
- **Dans un JAR** :
 - Le classpath peut être spécifié dans le manifeste via `Class-Path`.
 - Exemple :

```
Class-Path: lib1.jar lib2.jar
```

- **Remarque** : Lorsqu'on utilise `java -jar`, le classpath spécifié dans le manifeste est prioritaire. Les options `-classpath` ou la variable d'environnement `CLASSPATH` sont ignorées.

5. Commandes et options clés

- **Options de la commande `jar`** :
 - `c` : Créer une archive.
 - `v` : Afficher une sortie détaillée.
 - `f` : Spécifier le nom du fichier JAR.
 - `m` : Inclure un fichier manifeste.
 - `e` : Définir le point d'entrée (équivalent à `Main-Class`).
 - `t` : Lister le contenu du JAR.
 - `x` : Extraire le contenu du JAR.
- **Options de la commande `javadoc`** :
 - `-d <répertoire>` : Spécifier le répertoire de sortie.
 - `-version` : Inclure les tags `@version` dans la documentation.
 - `-author` : Inclure les tags `@author` dans la documentation.

```
javadoc -d MonJavaDoc geometry geometry.exceptions
```

6. Cas pratiques

- **Librairie** : Créer un JAR contenant des classes réutilisables.

```
jar cvf Transports.jar mTransports/*.class
```

- **Application exécutable** : Créer un JAR avec un manifeste contenant `Main-Class` et `Class-Path`.

```
jar cvfm Applic.jar manifeste.txt TestTransports.class
```

7. Résumé des bonnes pratiques

- **Pour les JAR exécutables** : Toujours spécifier `Main-Class` dans le manifeste.
- **Pour les dépendances** : Utiliser `Class-Path` dans le manifeste pour référencer d'autres JAR.
- **Portabilité** : Vérifier la version de la JVM pour éviter les erreurs d'exécution.

•